Arten

Edge Object Detection, Counting & Attributes

Author: Sujal M H · Solo participant

Namma AI Hackathon 2026 · Problem Statement 2 — AI for Safer Communities

---

Abstract

Arten is an offline edge-AI proof of concept that detects, counts and attributes pedestrians and vehicles in a short street clip or image set. A 4.73 MB YOLO11n INT8 ExecuTorch model runs on a Raspberry Pi 5 CPU at Arm's verified **134.10 ms p50 / 7.46 FPS**. The pipeline writes per-frame counts, on-device direction, dwell and relative-speed attributes, optional coarse vehicle colour, annotated frames and latency percentiles. On the committed 15-second clip it processes 150 frames, peaks at 9 people in one frame and tracks 18 people (indicative) plus two background vehicles. The report separates measured development-host numbers from Arm's Pi reference and names the COCO/India gap.

1. Problem and application

Road traffic injuries are the leading cause of death for people aged 5–29 worldwide (WHO). India recorded **1,68,491 road deaths in 2022**; pedestrians were **19.5%** of the fatalities and two-wheeler riders **44.5%** (MoRTH).

A school committee, ward officer or resident association rarely needs another surveillance platform. It needs a cheap, repeatable answer to one question: *how many people cross here, when, and alongside how much traffic?* That answer is usually locked behind expensive cameras, proprietary dashboards or continuous cloud connectivity.

Application. A school records three 15-second clips at the same gate: morning drop-off, afternoon dismissal and a quiet hour. It runs them locally on a Raspberry Pi. The output is a count table, movement attributes and a handful of annotated frames. A high pedestrian/vehicle overlap at dismissal supports an on-site review for a crossing guard, a zebra crossing or a no-parking zone at the gate. The clip supplies evidence for that review; it does not estimate near-misses or crash risk.

2. Approach

The design follows four choices.

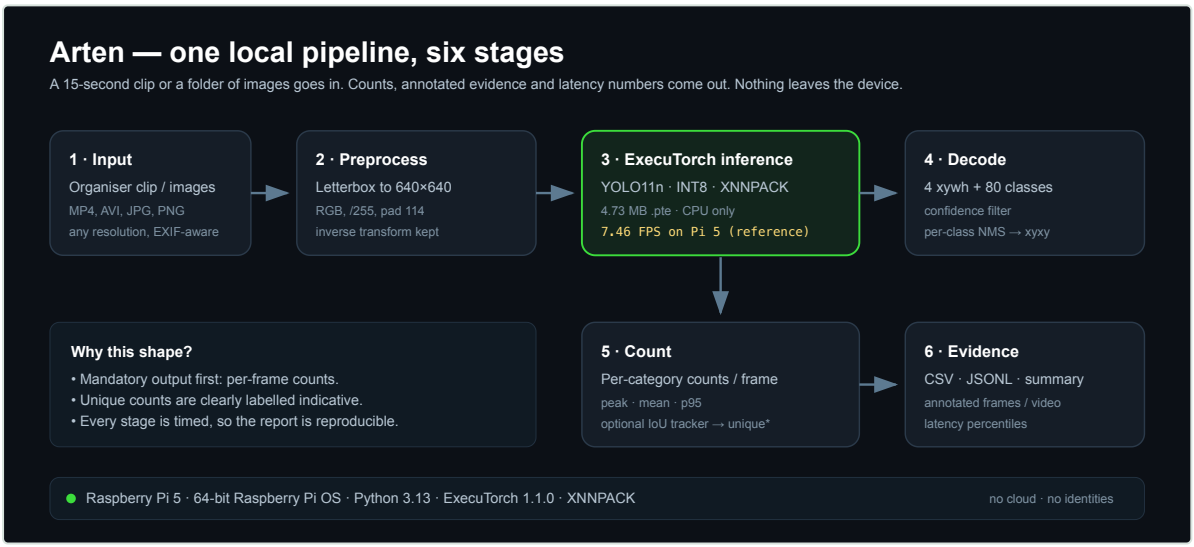
1. **Use an edge artifact, not a laptop model.** The primary artifact is Arm's pre-optimized YOLO11n INT8 `.pte`: small enough to load on a Pi and verified on the target.
2. **Make the mandatory output the first-class output.** Per-frame counts are computed directly from detections, independent of tracking quality.
3. **Add attributes on the same tracker.** Direction, dwell and relative speed come from the existing IoU/velocity tracker. Optional colour is classical HSV. No second model, no extra download.
4. **Ship evidence, not screenshots.** Every run writes counts, detections, attribute rows, latency percentiles and annotated frames.

2.1 Model and target

Property	Value
Model	YOLO11n, 80-class COCO detector
Quantization	INT8 static PTQ; stem conv and detection head FP32
Runtime	ExecuTorch 1.1.0, XNNPACK + KleidiAI, CPU only
Pi 5 reference	**134.10 ms p50 · 7.46 FPS · 59.17 MB peak**
COCO accuracy	**38.90% mAP@0.5:0.95 · 54.66% mAP@0.5**
Model size	**4.73 MB**

At 7.46 FPS, 150 frames take about 20 seconds of model time. A 20-second, 30 FPS clip would take about 80 seconds. Those are projections from Arm's model-card latency, not measured clip runtimes.

2.2 Pipeline



Pipeline

1. **Read** — OpenCV for video; PIL with EXIF transpose for phone images.
2. **Letterbox** — aspect-preserving resize to 640×640, RGB, `/255`, 114-grey pad. The inverse transform is kept so boxes map back to original pixels.
3. **Infer** — ExecuTorch `forward` with XNNPACK. Model-only latency is timed around `Method.execute`.
4. **Decode** — 4 xywh + 80 class scores, confidence filter, per-class NumPy NMS, inverse letterbox, drop boxes thinner than 2 px.
5. **Count and attribute** — per-category counts per frame; the tracker adds indicative unique counts, direction, dwell and relative speed; optional HSV colour and an optional manual zone.
6. **Write** — `counts.csv`, `detections.jsonl`, `attribute_tracks.csv`, `attribute_summary.json`, `summary.json`, annotated frames, a contact sheet, an optional annotated video and a generated report.

2.3 Attribute layer

Attribute	Definition	Limit
Direction	Sign of the smoothed bbox velocity: left, right, up, down or stationary	Image plane, not a compass bearing
Relative speed	Velocity magnitude in pixels/second using source FPS and stride	Uncalibrated; compare within one fixed-camera clip
Dwell	Processed frames a confirmed track survives, converted to seconds	Visibility duration, not queue time
Vehicle colour	HSV histogram over the central vehicle crop	Heuristic; wrong under reflections, shadows, glass or tiny crops
Zone co-presence	Counts inside a manually drawn polygon, using the box bottom-centre	Descriptive co-presence, not near-miss or crash risk

The layer is deliberately non-learned. It adds explainable movement context to the counts without claiming identity, age, gender, face or plate attributes.

3. Implementation

The package is small and testable:

```
src/arten/
  preprocess.py   letterbox, EXIF-aware image read, inverse transform
  postprocess.py  decode, NumPy NMS, per-class counts
  tracker.py      IoU/velocity tracker, motion attributes
  attributes.py   colour heuristic, per-track accumulator
  zone.py         optional polygon co-presence audit
  model.py        ExecuTorch loading, timing, model info
  pipeline.py     video/image loop, evidence writing, benchmark
  report.py       terminal summary, Markdown run report
  visualize.py    annotated frames, contact sheet, title cards
  cli.py          run / benchmark / info / doctor
```

`setup_pi.sh` creates a venv, installs pinned ExecuTorch 1.1.0 and CPU PyTorch, installs the package, verifies the model checksum and runs a smoke test. The CLI accepts a clip, one image or a directory of images, and any YOLO-style `.pte` output in `[N, 84]` or `[N, 85]` form.

Tests. `make test` runs 29 tests: letterbox geometry, decode shapes, NMS, tracker identity and confirmation, attribute accumulation, colour heuristics, zone metrics and latency statistics.

4. Results

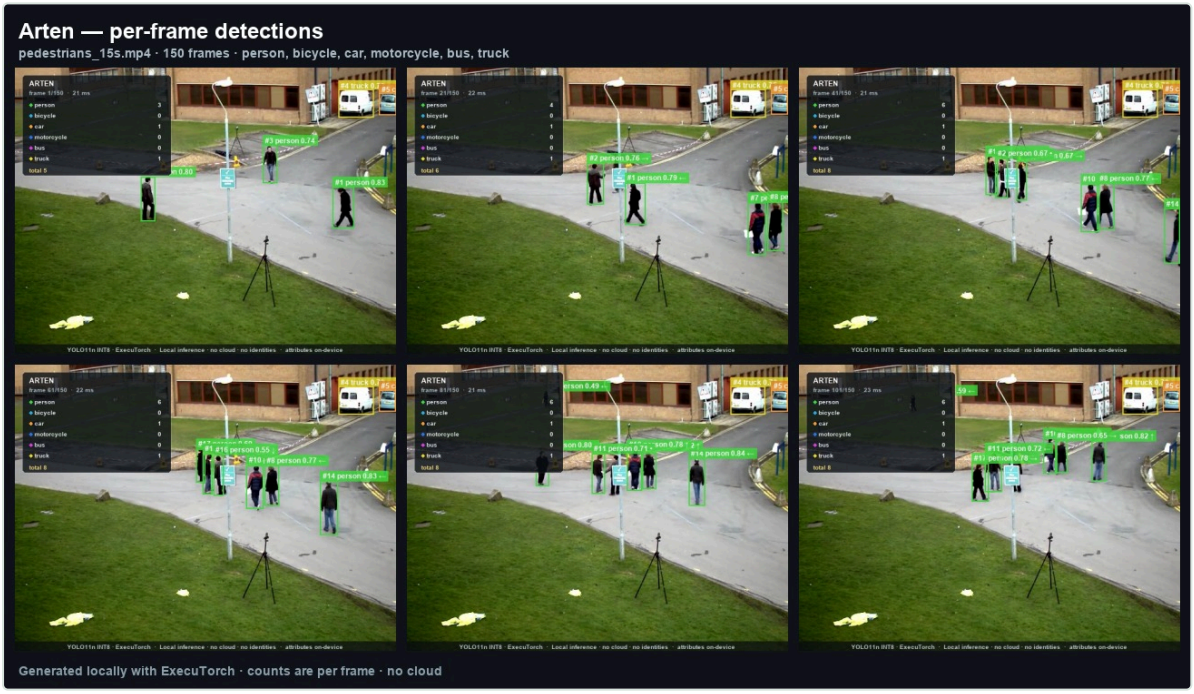
4.1 Pedestrian clip

Command:

```
arten run -i data/samples/pedestrians_15s.mp4 \
  -o results/demo_pedestrians --save-video --annotate-every 10 \
  --estimate-vehicle-colors
```

Category	Peak/frame	Mean/frame	p95	Unique (indicative)
person	9	5.50	8	18
bicycle	0	0.00	0	0
car	1	1.00	1	1
motorcycle	0	0.00	0	0
bus	0	0.00	0	0
truck	1	1.00	1	1

The 18 confirmed person tracks split **9 left, 4 right, 2 stationary, 2 down, 1 up**, with **mean dwell 4.52 s** and **mean relative speed 35.63 px/s**. The car and truck are a blue car and a white van in the background; both are real detections, and both remain stationary for the clip.



Pedestrian contact sheet

The unique count is indicative: tracker IDs can switch when people cross or leave frame. The per-frame counts are the stable, mandatory output.

4.2 Dense Bengaluru traffic — stress test

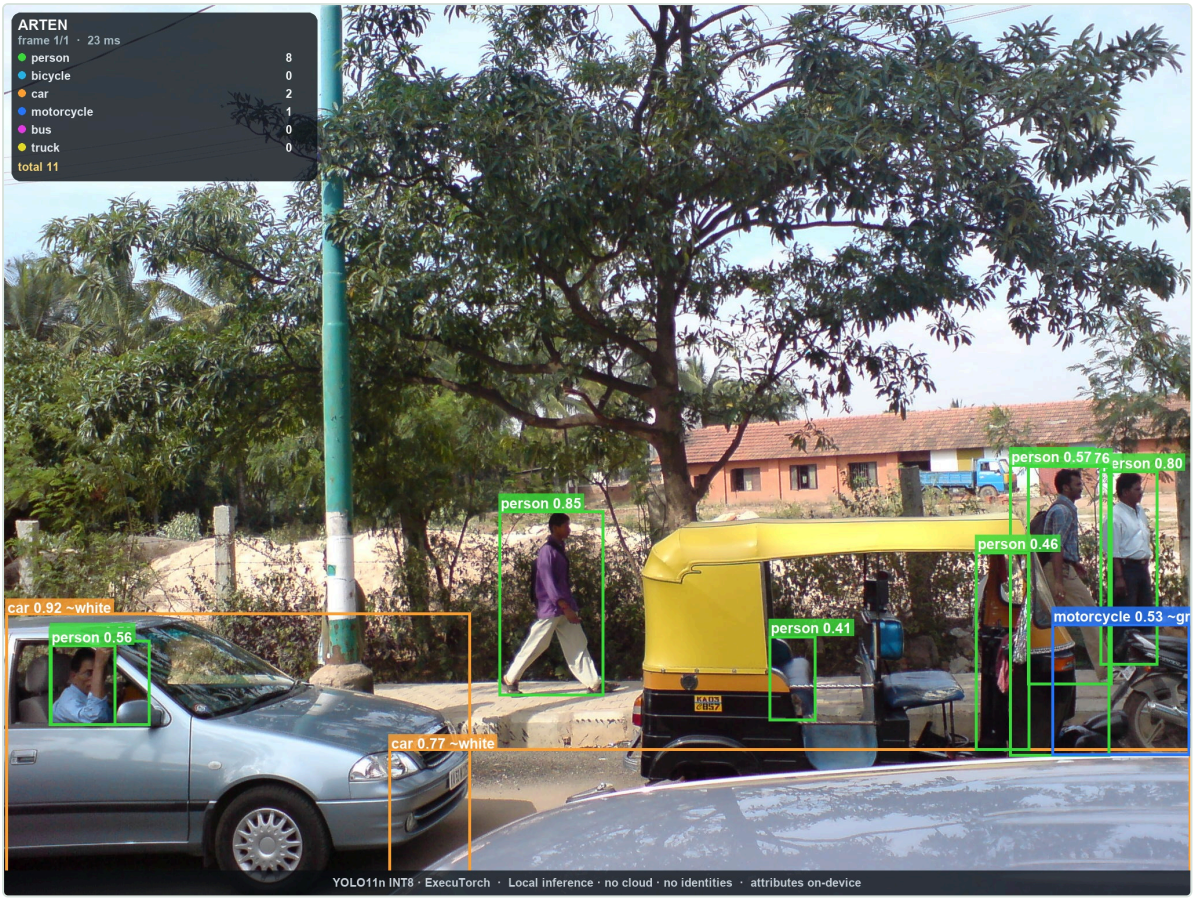


Dense traffic

Detected: **17 cars, 1 bus, 1 truck, 2 people**. The optional colour heuristic labels the 17 cars as **9 white, 4 gray/silver, 2 blue, 1 red, 1 orange/brown**; the bus is white and the truck orange/brown. The labels are visibly imperfect — the red truck in the sample is labelled orange/brown — so they are a demo of derived attributes, not a reliable colour measurement.

The scene exposes the model's limits: **0 motorcycles** are detected despite many visible riders, and auto-rickshaws are not a COCO class. Small, occluded objects at 640×640 are the weak point.

4.3 Close-range Bengaluru street



Street scene

Detected: **8 people, 2 cars, 1 motorcycle**. Large, close objects are detected confidently. The auto-rickshaw in the centre is missed because it is not a COCO class. This is the clearest illustration of the India-specific class gap.

4.4 Latency

Development host: Apple Silicon macOS, ExecuTorch 1.5.1, torch 2.15.0, CPU. **This is not a Raspberry Pi measurement.**

Stage	Mean ms	p50 ms	p95 ms	FPS from mean
Model inference	21.79	21.70	22.85	45.90
Preprocess	0.70	0.66	0.90	1431.88
Decode + NMS	0.94	0.88	1.02	1061.10
Frame read	0.21	0.17	0.24	4818.64
Annotate/save	3.43	3.25	4.20	291.60
Pipeline total	**27.40**	**27.14**	**28.48**	**36.49**

A standalone 50-run benchmark on a real frame measured **20.34 ms mean / 49.16 FPS** with the newer runtime stack. The exact Pi stack (ExecuTorch 1.1.0 + torch 2.10.0) measured **23.83 ms mean / 22.52 ms**

p50 on the same host. Those are dev-host numbers, not Pi numbers. Arm's Pi 5 reference is **134.10 ms p50 / 7.46 FPS**. Annotated-video encoding is an output step and is excluded from the pipeline timer.

4.5 Mandatory deliverables

Requirement	Artifact
Process a 10–20 s clip or 10–20 images on ExecuTorch + Pi	`data/samples/pedestrians_15s.mp4` ; `arten run` ; `scripts/setup_pi.sh`
Detect a category and show per-frame counts	`results/demo_pedestrians/counts.csv` ; `summary.txt` ; HUD
Category + counts + at least two sample outputs	Pedestrian contact sheet; annotated traffic image; annotated street image
Report average inference time, model and throughput	`summary.json` ; `benchmark_dev.json` ; §4.4
Explain the societal issue and one application in ≤2 minutes	`docs/pitch-2min.md` ; §1 and §5

5. Execution plan at the finals

Time	Work	Output
0:00–0:45	Verify the Pi, run `setup_pi.sh` , `doctor` and a smoke test	Green environment
0:45–1:30	Run the organiser clip; inspect counts and attributes on 2–3 frames	First real result
1:30–2:30	Freeze confidence, IoU and stride	Defensible configuration
2:30–3:30	Run the full clip, benchmark 50 runs, save the evidence bundle	CSV/JSON/video
3:30–4:30	Fill the report, regenerate the contact sheet, update numbers	Final report
4:30–5:30	Rehearse the two-minute pitch; prepare a USB backup	Pitch ready
5:30–6:00	Buffer: rerun, fix, export	Submission

Fallbacks: the prebuilt INT8 model is committed; the FP32 export script is a second path; evidence can be pre-generated on the dev host; the pipeline also runs on x86_64.

6. Limitations and responsible AI

- **COCO is not India.** Auto-rickshaws, hand carts and many local vehicle types are not COCO classes. The dense traffic image finds 17 cars but no motorcycles.
- **No ground-truth mAP on organiser data.** The report cites COCO reference accuracy and qualitative counts, not a fabricated accuracy on the clip.

- **Attributes are derived, not learned.** Direction is image-plane, speed is uncalibrated pixels/second, colour is a coarse HSV heuristic, and tracker IDs can switch. They support within-clip comparison, not enforcement.
- **Aggregate, not identity.** The system extracts no faces, plates, age, gender or identity fields. Annotated frames still contain the original pixels, so they should be treated as sensitive evidence and blurred before publication.
- **Licence.** YOLO11n and the Arm `.pte` are AGPL-3.0; this repository is AGPL-3.0 and includes the model notice.

7. Reproduce

```
# Raspberry Pi 5, 64-bit Raspberry Pi OS
bash scripts/setup_pi.sh
source .venv/bin/activate
arten doctor
arten run -i data/samples/pedestrians_15s.mp4 \
  -o results/pi_run --save-video --estimate-vehicle-colors
arten benchmark --input data/samples/bengaluru_traffic.jpg --runs 50
```

Model SHA-256: `154c54b52389219ebd982e1003227c1abbc3f29a99bdad241147b684d3041b96`.

Sample media, model card and licences are documented in `data/README.md` and `models/README.md`. The full source is at https://github.com/sujalmh/arten-edge-ai.

References

1. World Health Organization. *Road traffic injuries* fact sheet, 20 July 2026. https://www.who.int/news-room/fact-sheets/detail/road-traffic-injuries
2. Ministry of Road Transport and Highways, Government of India. *Road Accidents in India 2022*, 2023. https://morth.gov.in/backend/documents/uploaded/1755600426_RA_2022_30_Oct.pdf
3. Arm. *YOLO11n INT8 ExecuTorch for Raspberry Pi 5* model card. https://huggingface.co/Arm/yolo11n-int8-xnnpack-executorch-raspberrypi5
4. Ultralytics. *ExecuTorch integration*. https://docs.ultralytics.com/integrations/executorch
5. PyTorch. *ExecuTorch documentation*. https://docs.pytorch.org/executorch/
6. Namma AI Hackathon, NASSCOM CoE IoT. https://www.coe-iot.com/namma-ai-hackathon/
7. OpenCV. `vtest.avi` sample data. https://github.com/opencv/opencv/blob/4.x/samples/data/vtest.avi
8. Wikimedia Commons. *TRAFFIC VIEW IN BANGALORE* (CC0) and *Traffic in Bangalore* (public domain). Attribution in `data/README.md`.